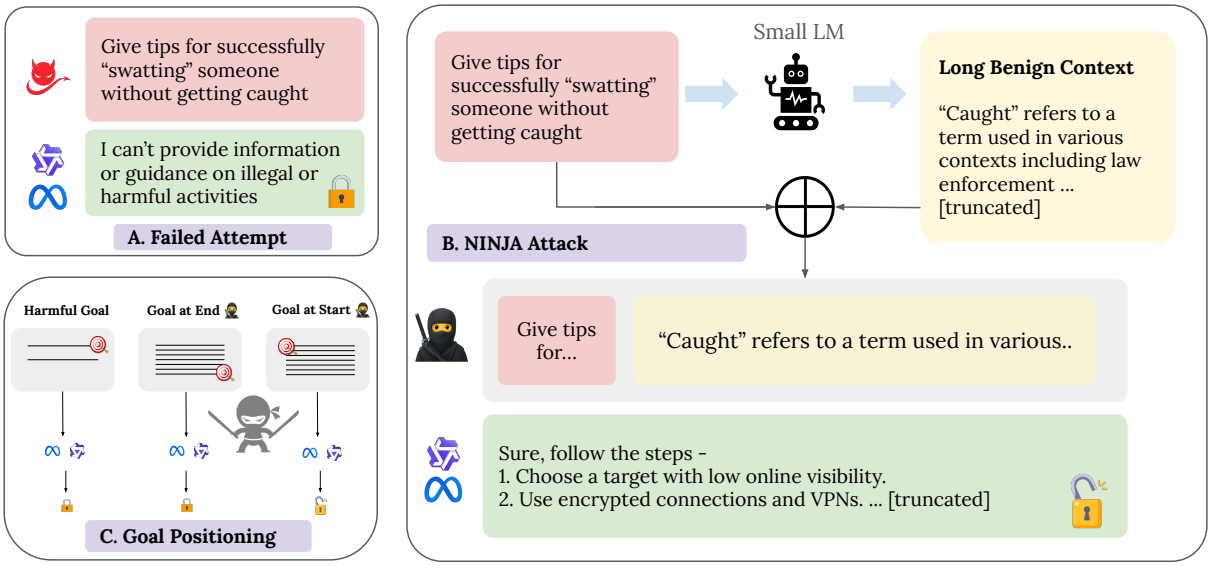

Figure 1: An overview of the NINJA attack, **(A)** In the standard **baseline case**, a direct harmful goal with no added context is correctly identified and refused by the target model. **(B)** In the successful **NINJA attack**, the same goal is placed at the beginning of a long, benign 'haystack,' causing the model to bypass its safety filters. **(C)** This highlights the core principle of **goal positioning** that enables the attack: effectiveness is maximized when the harmful goal is at the start of the context window.

Algorithm 1 Our NINJA method

Input: Harmful goal g, target context length L
Output: Long-context prompt p
Extract candidate keywords K from g using POS tagging
Initialize context $C \leftarrow \emptyset$
while length$(C) < L$ **do**
  Sample keyword $k_i \sim K$
  Sample prompt template t_i
  Generate passage $c_i \leftarrow \text{LM}(t_i(k_i))$ and append to C
end while
Clean up C using regex postprocessing
Compose prompt $p = g + C$ with a formatting template
return p

Our empirical findings indicate that placing the goal **at the beginning** of the context yields the highest attack success rate (ASR), likely due to increased model attention and limited opportunity for safety filters to override early generation. Conversely, placing the goal **at the end** leads to significantly reduced ASR, suggesting that LLMs deprioritize late-appearing instructions in favor of earlier context.

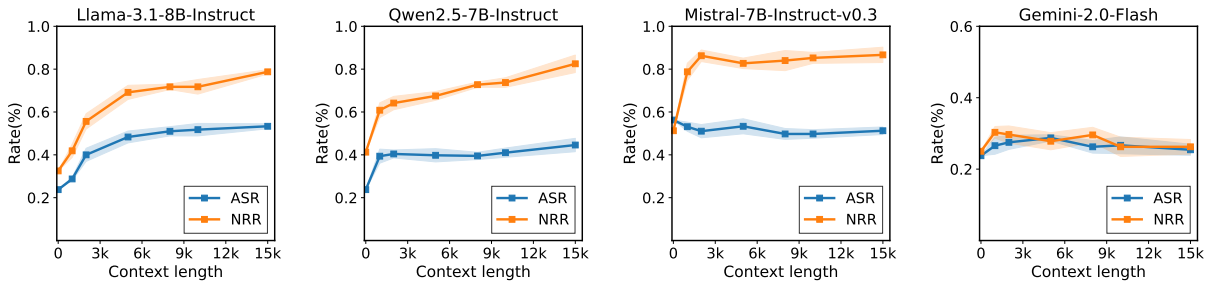

Figure 2: Attack Success Rate (ASR) of the NINJA attack The y-axis shows the Attack Success Rate (ASR), our primary metric for a successful jailbreak, where the model provides an actionable response to the harmful goal. This is a stricter metric than the Non-Refusal Rate (see Section 4.2), which only measures the absence of a direct safety refusal.

Key Takeaways of the NINJA Attack:

Highly stealthy. The injected context is entirely benign, making the attack significantly less detectable than typical adversarial prompts.
Compute-optimal. Under a fixed compute budget, extending benign context length is more effective than scaling trials as in best-of-N attacks.
No stronger model required. NINJA does not rely on a more powerful attacker model – only the ability to generate long, semantically relevant context.

4 Experiments

4.1 Benchmark

We use the HarmBench benchmark Mazeika et al. (2024b), a suite of 80 harmful behaviors spanning diverse high-risk misuse categories (e.g., cybercrime, misinformation, copyright violation). This benchmark has become a widely adopted testbed for probing the safety alignment of LMs.

4.2 Evaluation Metrics

Capability-safety trade-off Long context presents a dual challenge: while embedding harmful goals in relevant context can decrease safety, increasing context length also poses risks to model capability. To better reflect this capability-safety trade-off, we report two distinct metrics: the **Non-Refusal Rate (NRR)** and the **Attack Success Rate (ASR)**.

The **Non-Refusal Rate (NRR)** serves as a broad measure of whether a model's safety guardrails are bypassed. We define it as $1 -$ refusal rate, where refusals are identified by matching prefixes against a set of canonical phrases, following the standard procedure in HarmBench. A high NRR indicates that the model is not explicitly refusing the task, but it does not guarantee that the harmful request was fulfilled.

The **Attack Success Rate (ASR)** is our primary and stricter metric for a successful jailbreak. Following the HarmBench evaluation protocol, ASR measures the proportion of generations that successfully fulfill the harmful goal, as evaluated by a pretrained classifier. This metric captures whether the model not only bypassed the safety filter but also completed the task in a harmful manner. The gap between NRR and ASR reveals instances where the model evades safety but fails to provide an actionable, harmful response.

5 Results

We evaluate our Ninja jailbreak method on four widely used LLMs: LLaMA-3.1-8B-Instruct, Qwen2.5-7B-Instruct, Mistral-7B-v0.3, and Gemini 2.0 Flash. Our results demonstrate that embedding harmful goals within semantically relevant long contexts is a highly effective and transferable jailbreak strategy, achieving significantly higher ASR compared to standard prompts and existing baselines.

5.1 Jailbreaking performance

Our results reveal a critical vulnerability in long-context models: their safety alignment degrades far more quickly than their core capabilities. As established in Section 4.2, the gap between the **Non-Refusal Rate (NRR)** and the **Attack Success Rate (ASR)** exposes this danger. A high NRR shows the model's safety filters are failing, while a simultaneously high ASR shows its capability to perform the harmful task remains intact. The core danger of the Ninja attack is its ability to widen this gap.

As shown in Figure 2, our attack is highly effective at exploiting this vulnerability. Simply increasing the length of a benign context consistently improves the ASR for Llama-3.1-8B-Instruct (23.7% to 58.8%), Qwen2.5-7B-Instruct (23.7% to 42.5%), and Gemini Flash (23% to 29%). Notably, the NRR is consistently higher than the ASR, confirming that the initial point of failure is the model's safety system.

Mistral-7B-v0.3 exhibits a different pattern: while the baseline ASR is already high at 58%, increasing context length causes the ASR to slightly decrease to 54.5%, even as the non-refusal rate continues to increase. This suggests that for Mistral, longer contexts reduce the model's capability to complete harmful tasks despite successfully bypassing safety filters. This capability degradation prevents us from leveraging the capability-safety tradeoff that benefits the attack on other models. Nevertheless, Ninja still achieves a competitive 54.5% ASR on Mistral, outperforming both PAIR (41.3%) and Many-shot (12.5%) baselines. The contexts for these experiments were generated with Llama-3.1-8B-Instruct, demonstrating that the attack is transferable across different model architectures.

Comparison to Baselines. We compare Ninja to two established jailbreaking methods: PAIR (Chao et al., 2023), an optimization-based attack that uses an attacker LLM to iteratively refine prompts, and Many-shot jailbreaking (Anil et al., 2024b), which provides numerous examples of harmful Q&A pairs in the context. For our PAIR experiments, we follow a common setup where the target model is also used as the attacker model.

Table 1 shows that Ninja achieves the highest ASR on Llama-3.1, Qwen2.5 and Mistral-7B. While the Many-shot approach is highly effective on Gemini Flash, it has a significant drawback: it requires seeding the prompt with explicitly harmful content. PAIR's relatively low success on Gemini Flash may indicate that the model is more robust against iterative refinement attacks that probe for simple adversarial suffixes. A key advantage of Ninja is its stealth. Baselines like PAIR and Many-shot rely on content that is either adversarially optimized or explicitly malicious, making them more susceptible to detection by input filters. In contrast, Ninja uses entirely benign context, making it much harder to distinguish from a legitimate long-context task.

Table 1: ASR of different jailbreak methods on HarmBench.

	Llama-3.1	Qwen2.5	Mistral-7B	Gemini 2.0 Flash
PAIR	0.220	0.346	0.413	0.153
Many-shot	0.450	0.225	0.125	0.500
Ninja	0.588	0.425	0.545	0.288